**Assignment 1: Create a simple login page in Django where users can log in, see a dashboard, and log out.**

**Step 1: Create Django Project**

1. Open your terminal or PowerShell.
2. Go to the folder where you want your project, e.g., Desktop:

cd C:\Users\Sai\Desktop

3. Create Django project:

django-admin startproject mysite

**Meaning:**

- mysite is your project folder.
- Django will create files like manage.py, settings.py, urls.py, and a folder mysite/.

4. Move into your project folder:

cd mysite

**Output:**

- Your terminal path changes to C:\Users\Sai\Desktop\mysite.

---

**Step 2: Create an App**

1. Create an app named accounts:

python manage.py startapp accounts

**Meaning:**

- accounts will hold our login logic (views, templates, URLs).
- It creates a folder accounts/ with files: views.py, models.py, apps.py, etc.

2. Add app to INSTALLED_APPS in mysite/settings.py:

```
INSTALLED_APPS = [
    ...
    'accounts', ]
```

**Meaning:**

- Django knows to use our accounts app.

---

**Step 3: Create Templates Folder**

1. Inside accounts app, create folders:

```
accounts/    templates/
accounts/
login.html
dashboard.html
```

**Meaning:**

- Django looks inside templates for HTML pages.
- The second accounts folder is **namespace** to avoid conflicts if multiple apps have same file names.

---

**Step 4: Create login.html**

Inside accounts/templates/accounts/login.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>Login Page</title>
</head>
<body>
    <h2>Login</h2>
```

```
  {% if form.errors %}
    <p style="color:red;">Invalid username or password</p>    {%
endif %}

  <form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Login</button>
  </form>
</body> </html>
```

**Meaning:**

- {% csrf_token %} protects against security attacks.
- {{ form.as_p }} shows the login form fields.
- If login fails, an error message appears.

---

**Step 5: Create dashboard.html**

Inside accounts/templates/accounts/dashboard.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>Dashboard</title>
</head>
<body>
  <h2>Welcome {{ user.username }}</h2>
  <p>You have successfully logged in.</p>

  <a href="{% url 'logout' %}">Logout</a>
```

</body> </html>

**Meaning:**

- {{ user.username }} displays logged-in username.
  Logout link logs the user out.

---

**Step 6: Create views.py** Inside

accounts/views.py:

```
from django.shortcuts import render, redirect

from django.contrib.auth.decorators import login_required


def home(request):

    return redirect('login')  # Redirect / → /login/


@login_required def dashboard(request):     return
render(request, 'accounts/dashboard.html')
```

**Meaning:**

- home redirects user to login page.
- dashboard can only be accessed by logged-in users (@login_required).

---

**Step 7: Create urls.py in accounts**

Inside accounts/urls.py:

```
from django.urls import path from .
import views

from django.contrib.auth import views as auth_views


urlpatterns = [
```

```
    path('', views.home, name='home'),  # / → login

    path('login/', auth_views.LoginView.as_view(template_name='accounts/login.html'),
name='login'),

    path('logout/', auth_views.LogoutView.as_view(), name='logout'),
path('dashboard/', views.dashboard, name='dashboard'),

]
```

**Meaning:**

- /login/ uses Django built-in LoginView with our login.html template.
- /logout/ logs out the user.
- /dashboard/ shows dashboard page.

---

**Step 8: Include accounts URLs in project**

Inside mysite/urls.py:

```
from django.contrib import admin

from django.urls import path, include


urlpatterns = [

    path('admin/', admin.site.urls),

    path('', include('accounts.urls')),  # Include accounts app URLs

]
```

**Meaning:**

- All URLs starting with / go to accounts.urls.

---

**Step 9: Update settings.py for login/logout redirects**

Inside settings.py add at the bottom:

```
# Redirect after login
```

LOGIN_REDIRECT_URL = '/dashboard/'

# Redirect after logout

LOGOUT_REDIRECT_URL = '/login/'

**Meaning:**

- After successful login → dashboard
- After logout → login page

---

**Step 10: Apply migrations and create superuser**

1. Apply migrations:

python manage.py makemigrations python
manage.py migrate

2. Create a superuser:

python manage.py

createssuperuser

- Enter username: mca
- Enter password
- Confirm password
  **Output:** Superuser created successfully.

**Meaning:**

- This user will log in for the first time.

---

**Step 11: Run the server** python

manage.py runserver

**Output:** Server starts at http://127.0.0.1:8000/

---

**Step 12: Test in browser**

1. Open browser: http://127.0.0.1:8000/login/
2. Enter username: mca
3. Enter password (the one you set)
4. Click **Login** → Redirects to http://127.0.0.1:8000/dashboard/
5. You see:

Welcome mca

You have successfully logged in.

Logout

6. Click **Logout** → Back to login page

✅ **Output works perfectly**

---

**Step 13: Explanation of Folder Structure**

mysite/

|

├── manage.py

├── mysite/ | ├── settings.py | ├── urls.py

| └── wsgi.py

|

├── accounts/

| ├── views.py

| ├── urls.py

| ├── templates/

| └── accounts/

```
|        ├── login.html
|        └── dashboard.html
| └── __init__.py
```

- accounts/views.py → contains login logic
- accounts/urls.py → contains URL paths
- accounts/templates/accounts/ → contains HTML pages
- mysite/urls.py → connects app URLs to project

**Assignment 2: To design a Django web application using Django Templates to display HTML GUI for the client.**

Procedure

1. Create Project & Activate Virtual Environment

```
django-admin startproject mysite
cd mysite
python -m venv venv
venv\Scripts\activate
```

2. Create Apps

```
python manage.py startapp accounts
python manage.py startapp webapp2
```

3. Register Apps

File: mysite/settings.py

```
INSTALLED_APPS = [
...
'accounts',
'webapp2',
]
```

4. Create Template Folder

Inside webapp2 create folders:

```
webapp2
└── templates
└── home.html
```

5. Write View

File: webapp2/views.py

```
from django.shortcuts import render
from django.contrib.auth.decorators import login_required
@login_required
def home(request):
return render(request, 'home.html', {'name': request.user.username})
```

6. Create URLs

File: webapp2/urls.py----------On webapp2 app right click and create new file urls.py

```
from django.urls import path
from . import views
urlpatterns = [
path('template/', views.home),
]
```

File: mysite/urls.py----------Only Add path of new app

```
from django.contrib import admin
from django.urls import path, include
urlpatterns = [
path('admin/', admin.site.urls),
path('', include('accounts.urls')),
path('webapp2/', include('webapp2.urls')),
]
```

7. Write Template

File: home.html

```
<html>
<body>

<h2>Welcome {{ name }}</h2>
<p>You have successfully logged in.</p>
<a href="/logout/">Logout</a>
</body>
</html>
```

8. Run Server

python manage.py runserver

Open in browser:

http://127.0.0.1:8000/template/

Output

Welcome saii

You have successfully logged in.

Logout

Result

Thus, Django Template is successfully used to display the HTML GUI for the client.

***ORRRRRRRR***

Create Super User (Admin Login)

python manage.py createsuperuser

Meaning:

Creates an admin user to login into Django system.

Open Django Admin Panel

python manage.py runserver

Then open:

http://127.0.0.1:8000/admin/ Login using the superuser account.

**Assignment 3: Create a simple student information page using Django templates.**

---

**Step 1: Create App**

python manage.py startapp students

Add in settings.py

'students',

---

**Step 2: Create URL**

**students/urls.py**

```
from django.urls import path
from . import views

urlpatterns = [
    path('student/', views.student_info, name='student'),
]
```

Add in main urls.py

path('', include('students.urls')),

---

**Step 3: Create View**

**students/views.py**

```
from django.shortcuts import render

def student_info(request):
    data = {
        'name': 'Rahul',
        'course': 'MSc Computer Application',
        'marks': 85
    }
    return render(request, 'student.html', data)
```

---

**Step 4: Create Template**

Create:

templates/student.html

**student.html**

```
<!DOCTYPE html>
<html>
<head>
   <title>Student Info</title>
</head>
<body>
   <h1>Student Information</h1>
   <p>Name: {{ name }}</p>
   <p>Course: {{ course }}</p>
   <p>Marks: {{ marks }}</p>

   {% if marks > 50 %}
      <p style="color:green;">Result: Pass</p>
   {% else %}
      <p style="color:red;">Result: Fail</p>
   {% endif %}
</body>
</html>
```

---

**Run Server**

python manage.py runserver

Open:

http://127.0.0.1:8000/student/

---

**Output**

Student Information
Name: Rahul
Course: MSc Computer Application
Marks: 85
Result: Pass

**Assignment 4: Create a Student model and perform create, insert and update operations using Django ORM with PostgreSQL database.**

---

**Step 1: Install & Setup PostgreSQL**

Make sure PostgreSQL is installed.

Create Database:

```
CREATE DATABASE college_db;
```

Note: You must also install:

```
pip install psycopg2
```

---

**Step 2: Configure Database in Django**

File: settings.py

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'college_db',
        'USER': 'postgres',
        'PASSWORD': 'your_password',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

---

**Step 3: Create App**

```
django-admin startproject college
cd college
python manage.py startapp students
```

Add app in settings.py:

```
INSTALLED_APPS = [
    ...
    'students',
]
```

---

**Step 4: Create Model**

File: students/models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    course = models.CharField(max_length=100)

    def __str__(self):
        return self.name
```

---

**Step 5: Apply Migrations (Create Table)**

```
python manage.py makemigrations
python manage.py migrate
```

Table created in PostgreSQL automatically using ORM.

---

**Step 6: Insert & Update Data using Django Shell**

```
python manage.py shell
```

**Insert Data**

```
from students.models import Student

s1 = Student.objects.create(name="Rahul", age=21, course="BCA")
s2 = Student.objects.create(name="Amit", age=22, course="MCA")
```

**Update Data**

```
student = Student.objects.get(name="Rahul")
student.age = 23
student.save()
```

---

**Step 7: Check Output**

```
Student.objects.all()
```

**Output:**

```
<QuerySet [<Student: Rahul>, <Student: Amit>]>
```

In PostgreSQL GUI (pgAdmin), table shows:

| id | Name | age | course |
|---|---|---|---|
| 1 | Rahul | 23 | BCA |
| 2 | Amit | 22 | MCA |

**Assignment 5: Create table and insert records using pgAdmin GUI and retrieve data using Django ORM.**

---

**Step 1: Open pgAdmin (GUI of PostgreSQL)**

Use:

- pgAdmin 4

Create table manually:

Table Name: teacher

Columns:

- id (Primary Key, Auto Increment)
- name (varchar)
- subject (varchar)

*ORRR*

**Using SQL Command**

**Steps:**

1. Right-click your database
2. Click **Query Tool**
3. Write this SQL command:

```
CREATE TABLE teacher (
   id SERIAL PRIMARY KEY,
   name VARCHAR(100),
   subject VARCHAR(100)
);
```

4. Click **Execute (▶ button)**

Table will be created.

---

**Step 2: Create Model in Django**

File: students/models.py

```
class Teacher(models.Model):
    name = models.CharField(max_length=100)
    subject = models.CharField(max_length=100)

    def __str__(self):
        return self.name
```

---

**Step 3: Fetch Data using ORM**

```
python manage.py shell
from students.models import Teacher
Teacher.objects.all()
```

**Output:**

```
<QuerySet [<Teacher: Raj>, <Teacher: Priya>]>
```

---

**Assignment 6: Create a view that fetches student data from database and display using URL routing.**

---

**models.py**

students/models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    course = models.CharField(max_length=100)

    def __str__(self):
        return self.name
```

---

**views.py**

students/views.py

```
from django.shortcuts import render
from .models import Student

def student_list(request):
    students = Student.objects.all()
    return render(request, 'student_list.html', {'students': students})
```

---

**urls.py**

students/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
```

```
    path('students/', views.student_list, name='student_list'),
]
```

---

**Project urls.py**

college/urls.py

```
from django.urls import path, include

urlpatterns = [
    path('', include('students.urls')),
]
```

---

**Template File**

students/templates/student_list.html

```
<h2>Student List</h2>

{% for student in students %}
    <p>{{ student.name }} - {{ student.course }}</p>
{% endfor %}
```

---

**Run Server**

```
python manage.py runserver
```

Open Browser:

http://127.0.0.1:8000/students/

---

**Output on Browser:**

Student List

Rahul - BCA
Amit - MCA

---

**Assignment 7: Create a basic addition program using Django form and display result.**

---

views.py

```
def add(request):
    if request.method == "POST":
        a = int(request.POST['a'])
        b = int(request.POST['b'])
        return HttpResponse(f"Addition is {a + b}")
    return render(request, 'add.html')
```

---

urls.py

```
path('add/', views.add),
```

---

**Output:**

Addition is 30

**Assignment 8: Create a Student Registration Form using an HTML form in Django.**
**The form should accept: Student Name, Age and Course. After submitting the form, the data should be stored in the database.**

---

**Project Structure**

Assume:

- Project Name → myproject
- App Name → studentapp

```
myproject/
│
├── myproject/
│
├── studentapp/
│   ├── models.py
│   ├── views.py
│   ├── urls.py
│   ├── templates/
│   │   └── register.html
│
└── manage.py
```

---

**models.py**

studentapp/models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    course = models.CharField(max_length=100)

    def __str__(self):
        return self.name
```

---

**views.py**

studentapp/views.py

```
from django.shortcuts import render
from .models import Student

def register(request):
    if request.method == "POST":
        name = request.POST['name']
```

```
        age = request.POST['age']
        course = request.POST['course']
        Student.objects.create(name=name, age=age, course=course)

    return render(request, 'register.html')
```

---

**urls.py**

studentapp/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('register/', views.register, name='register'),
]
```

---

**urls.py**

myproject/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('studentapp.urls')),
]
```

---

**register.html**

studentapp/templates/register.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
</head>
<body>
    <h2>Student Registration Form</h2>
```

```
    <form method="POST">
        {% csrf_token %}

        Name: <input type="text" name="name"><br><br>
        Age: <input type="number" name="age"><br><br>
        Course: <input type="text" name="course"><br><br>

        <button type="submit">Submit</button>
    </form>

</body>
</html>
```

**How to Run the Program**

**Step 1: Create Migrations**

python manage.py makemigrations

**Step 2: Apply Migrations**

python manage.py migrate

**Step 3: Run Server**

python manage.py runserver

**Step 4: Open Browser**

http://127.0.0.1:8000/register/

---

**Output**

**Output Screen 1 (Form Page)**

```
----------------------------
Student Registration Form

Name:   [___________]
Age:    [___________]
Course: [___________]

[ Submit ]
----------------------------
```

**After Clicking Submit:**

- Data is saved in database.
- Page reloads.
- You can verify data in:

http://127.0.0.1:8000/admin/

**Optional: admin.py (To See Data in Admin Panel)**

studentapp/admin.py

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

---

**Result**

✓ Student details stored in database
✓ Form loads properly
✓ No errors while submitting

**Assignment 9: Create a Student Registration form using Django ModelForm.**
**The form should: Accept name, age, and course, Validate the data, Save data into database**

---

**Project Structure**

App Name: studentapp

```
studentapp/
│
├── models.py
├── forms.py
├── views.py
├── urls.py
├── templates/
│   └── form.html
```

---

**models.py**

studentapp/models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    course = models.CharField(max_length=100)

    def __str__(self):
        return self.name
```

---

**forms.py**

studentapp/forms.py

```
from django import forms
from .models import Student

class StudentForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ['name', 'age', 'course']
```

---

**views.py**

studentapp/views.py

```
from django.shortcuts import render
from .forms import StudentForm
```

```
def form_register(request):
    form = StudentForm(request.POST or None)
    if form.is_valid():
        form.save()

    return render(request, 'form.html', {'form': form})
```

---

**urls.py**

studentapp/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('form-register/', views.form_register, name='form_register'),
]
```

---

**form.html**

studentapp/templates/form.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Form Class Registration</title>
</head>
<body>
    <h2>Student Registration using ModelForm</h2>

    <form method="POST">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Submit</button>
    </form>

</body>
</html>
```

---

▶ **How to Run**

```
python manage.py makemigrations
python manage.py migrate
python manage.py runserver
```

Open:

http://127.0.0.1:8000/form-register/

---

**Output**

✓ Form displayed
✓ Validation works automatically
✓ Data saved in database
✓ Errors shown if field is empty

---

**Assignment 10: Create an API using Django REST Framework that returns all student data in JSON format.**

---

**Install Package**

```
pip install djangorestframework
```

Add in settings.py:

```
INSTALLED_APPS = [
    ...
    'rest_framework',
]
```

---

**serializers.py**

studentapp/serializers.py

```
from rest_framework import serializers
from .models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = '__all__'
```

---

**views.py**

studentapp/views.py

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Student
from .serializers import StudentSerializer

@api_view(['GET'])
def student_api(request):
    students = Student.objects.all()
    serializer = StudentSerializer(students, many=True)
    return Response(serializer.data)
```

---

**urls.py**

```
path('student-api/', views.student_api, name='student_api'),
```

---

**▶ How to Run**

```
python manage.py runserver
```

Open:

http://127.0.0.1:8000/student-api/

---

**Output (JSON Format)**


```
[
  {
    "id": 1,
    "name": "John",
    "age": 20,
    "course": "BSc"
  }
]
```


✓ API working
✓ Data in JSON format

---

**Assignment 11: Initialize a Git repository for a Django project and perform the following operations:**

1. **Initialize Git repository**
2. **Add project files**
3. **Commit files**
4. **Check repository status**
5. **View commit history**

---

**Software Required:**

- Git installed in system
- Django project folder (example: myproject)

---

**Where to Run Git Commands?**

You can run Git commands in:

✓ Windows Command Prompt (CMD)
✓ PowerShell
✓ Git Bash
✓ VS Code Terminal (Recommended)

**VS Code Terminal inside Django project folder**

---

**STEP 1: Open Django Project Folder**

Example:

C:\Users\Swami\Desktop\myproject

Open this folder in VS Code.

Then open terminal:

Terminal → New Terminal

---

**STEP 2: Initialize Git Repository**

▶ **Command:**

```
git init
```

**Output:**

```
Initialized empty Git repository in C:/Users/Swami/Desktop/myproject/.git/
```

✓ This creates hidden .git folder
✓ Now your project is under version control

---

**STEP 3: Check Status**

▶ **Command:**

```
git status
```

**Output:**

```
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        manage.py
        myproject/
        myapp/
```

✓ Shows all untracked files

---

**STEP 4: Add Files to Staging Area**

▶ **Command:**

```
git add .
```

. means add all files

---

▶ **Check Status Again:**

```
git status
```

**Output:**

```
Changes to be committed:
      new file: manage.py
      new file: myproject/settings.py
      new file: myapp/models.py
```

✓ Files are now staged

---

**STEP 5: Commit Files**

▶ **Command:**

```
git commit -m "First Commit"
```

---

**STEP 6: View Commit History**

▶ **Command:**

```
git log
```

**Output:**

```
commit 4f3a2bc8f6e12345abcde6789
Author: Swami <swami@gmail.com>
Date:   Sun Feb 23 10:30:00 2026

    First Commit
```

✓ Shows commit ID
✓ Author name
✓ Date
✓ Commit message

---

**Optional: Configure Git (First Time Only)**

If commit gives error, run:

```
git config --global user.name "Swami"
git config --global user.email "swami@gmail.com"
```

Then commit again.

---

**Final Folder Structure After Git Init**

```
myproject/
│
├── .git/
├── manage.py
├── myproject/
├── myapp/
```

.git folder is created automatically.

---

**Result:**

✓ Git repository initialized
✓ Files added
✓ Files committed
✓ Commit history viewed
✓ Version control successfully implemented

**Assignment 12: Perform Create, Read, Update, Delete operations using Django ORM.**

---

**Write Code in Django Shell**

Run:

python manage.py shell

---

**CRUD Code**

**Create**

```
from studentapp.models import Student
Student.objects.create(name="John", age=20, course="BSc")
```

---

**Read**

```
Student.objects.all()
```

Output:

```
<QuerySet [<Student: John>]>
```

---

**Update**

```
student = Student.objects.filter(id=1).first()
student.name = "Updated Name"
student.save()
```

---

**Delete**

```
student.delete()
```

---

**Final Output**

✓ Record created
✓ Record displayed
✓ Record updated
✓ Record deleted